Smart Lighting and Independent Plant Monitoring

Technical report draft • structured to match the supplied IoT laboratory report template

1. Cover information

Field	To complete
Course / experiment	[Insert course name] / Course project
Group number	[Insert group number]
Submission date	[Insert date]
Members and contributions	[Insert full names and individual contributions]
Repository	github.com/fargekatten/Water-and-light-system
Development baseline	codex/thingspeak-talkback; last confirmed commit 45a67b5
Final source status	Local additions and edits demonstrated; final commit not yet captured

Document basis: user-supplied code, terminal output, dashboard text, and physical test confirmations in the project session. The author of this draft did not independently inspect the final Mac files or repeat the physical experiments. Confirmed observations and proposed tests are distinguished throughout.

2. Objective

Design and demonstrate an Internet-connected prototype that controls domestic Hue lighting from activity, screen-position and ambient-light sensors, accepts remote manual commands, and displays independent soil-moisture measurements. The lighting node should communicate through a mobile hotspot, apply defined rule priorities, and retain the current light output when sensor data become stale. Moisture monitoring must not control watering or lighting.

Abstract

Two Arduino Uno WiFi Rev2 boards publish to separate ThingSpeak channels using HTTPS. A Mac polls lighting data, evaluates a priority policy, and controls a Hue Bridge. A separate authenticated local web interface displays light status and raw soil moisture. Physical observations confirmed warm lighting from sofa activity, cinema lighting from screen detection after adjusting the threshold, actual Hue switching, soil telemetry, and end-to-end operation through a mobile hotspot. Long-duration timeout tests and quantitative response-time measurements remain to be documented.

3. System overview

Figure 1 shows two independent telemetry paths. Node 1 publishes sofa and screen analog values, ambient light and touch to a lighting channel. Node 2 publishes raw soil moisture to a private plant channel. The Mac reads Node 1 every five seconds and uses a shared Hub to arbitrate automatic and manual commands. The Hue Bridge operates lamps 4, 5 and 6. The school demonstration was validated using the phone hotspot for Node 1 while the Mac and lamps remained at home.

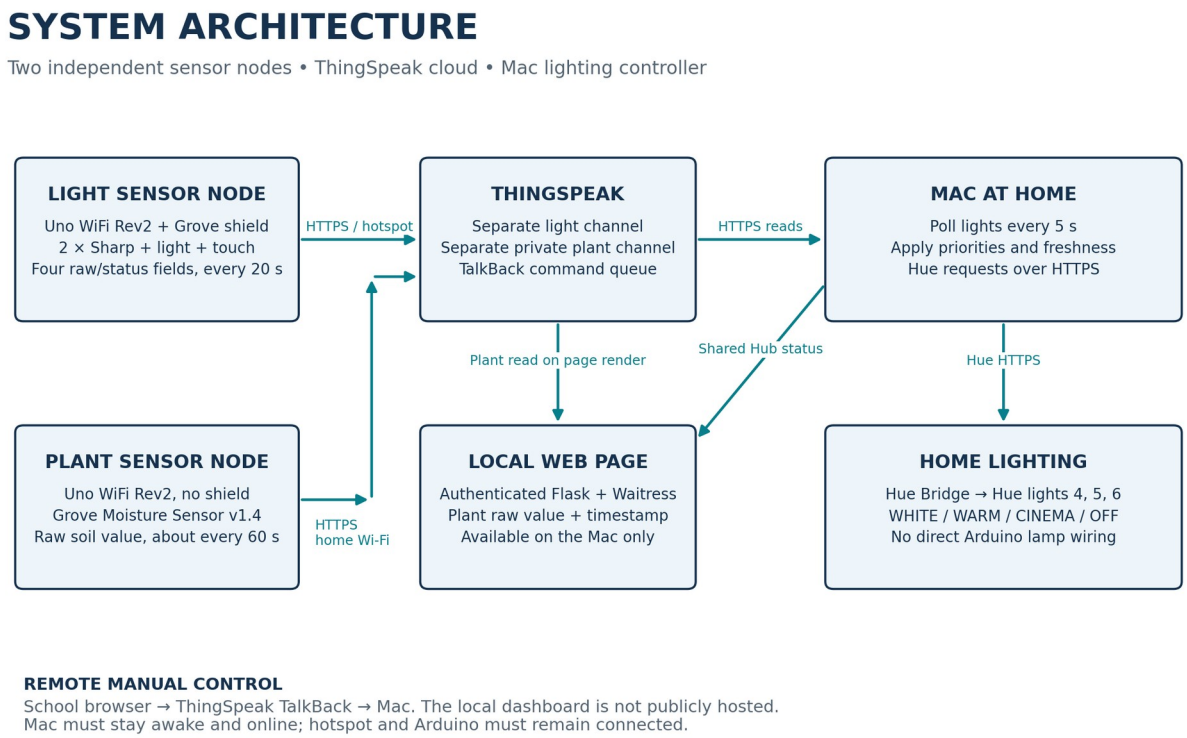


Figure 1. Final demonstrated architecture and data/control paths. Remote dashboard hosting was not implemented.

The dashboard is served on 127.0.0.1:8765 and is accessible on the Mac itself. Remote manual operation uses ThingSpeak TalkBack in a browser; the dashboard does not become remotely accessible merely because telemetry is cloud-hosted. Node 2 remains on home Wi-Fi and can run from a USB charger without a PC.

4. Hardware implementation

Component	Role / interface
2 × Arduino Uno WiFi Rev2	Separate lighting and plant nodes; WiFiNINA networking
Grove shield on Node 1	Grove sensor connection; exact shield revision to record
2 × Sharp analog distance sensors	Sofa and screen; marking consistent with GP2Y0A21YK0F (10–80 cm nominal)
Grove analog light + Grove touch	Ambient-light input and manual activity input
Grove Moisture Sensor v1.4	Resistive soil measurement; raw values only
Breadboard, leads, USB supplies	Sharp power distribution and low-voltage connections
Hue Bridge + lights 4, 5, 6	Colour-temperature lighting; no RGB orange requirement
Mac, Arduino IDE PC, smartphone	Controller, firmware programming, hotspot

Table 1. Hardware inventory. Confirm exact model markings and shield revision from the assembled prototype.

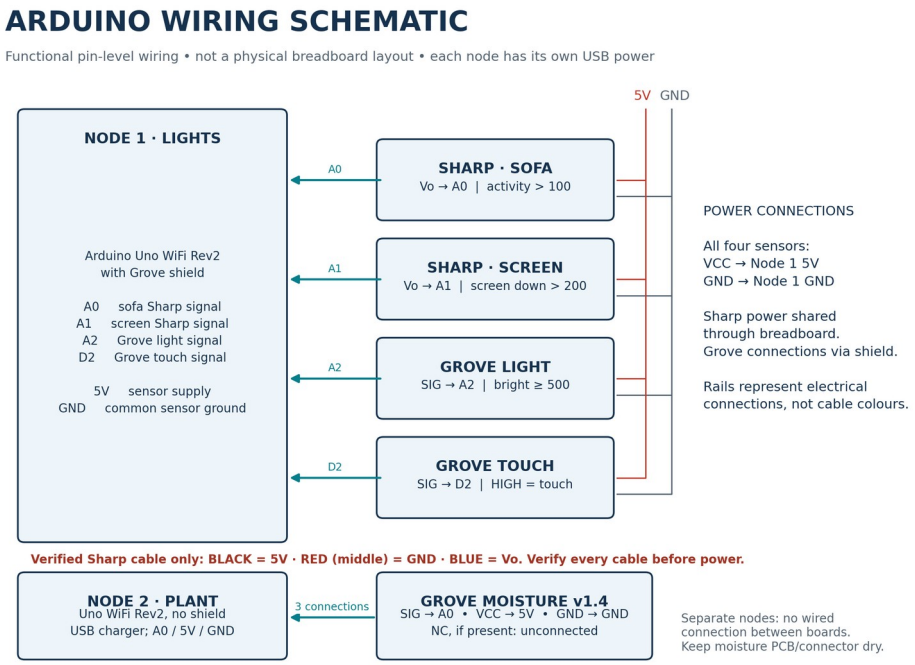


Figure 2. Functional wiring schematic. The signal, supply and common-ground connections are specified; the physical shield/breadboard layout must be documented in a photograph.

Both nodes use 5 V sensor supply and their own USB power. No lamp, pump, relay, transistor driver or flyback diode is connected to the Arduino. No added series resistors or signal protection were reported; inspect the final build before submission. On the verified Sharp cable, black was VCC, red (middle) GND and blue analog output. A previous colour-based assumption caused upload failure until corrected. Never generalise these colours to other cables. Keep the plant sensor PCB and connector dry.

5. Software and communication

The final local lighting sketch uses WiFiNINA, ThingSpeak and WiFiSSLClient with TS_ENABLE_SSL. It reads A0/A1/A2 and D2, retries Wi-Fi at nominal 10 s intervals, and attempts a channel update every 20 s. Touch is latched until a successful HTTP 200 write or a 30 s expiry; network calls can block sensor sampling. The plant sketch reads A0 and writes Field 1 approximately every 60 s, reconnecting as needed.

Light channel field	Interpretation
1	Sofa raw; activity when >100
2	Screen raw; screen down when >200 (revised from >400)
3	Light raw; bright when ≥500
4	Touch pending, 0 or 1; contributes to activity

Table 2. Final ThingSpeak field mapping and thresholds. Plant channel Field 1 is independent raw moisture.

Priority	Condition / output
1	Freshness gate: absent or >75 s old → no new Hue command
2	Screen down → CINEMA, warm-white 25%; cancels manual override
3	Active manual override → WHITE / WARM / CINEMA / OFF, up to 15 min
4	Activity within last 30 min → WARM, 50%
5	Otherwise 07:00–23:00 Europe/Oslo and dark → WHITE, 100%
6	Otherwise → OFF

Table 3. Controller decisions. Screen priority applies when data are fresh; stale data retain the previous physical light output.

```
motion = int(sofa > 100 or touch == 1)
screen_down = int(screen > 200)
light_bright = int(light >= 500)
```

thingspeak_lights.py validates fields and timestamps. thingspeak_controller.py processes each fresh entry only once, converts cloud measurement age to a monotonic observation time, and uses Hub / LightingPolicy. Repeated polling of an old entry does not renew activity or freshness. The same 75 s timeout replaces the former 3 s checks in lighting.py and hub.py. The controller checks desired output about every 0.5 s; run_controller checks every 0.1 s and sends changed states. HTTP and Hue latency add to these intervals.

WHITE uses brightness 254 and CT 153; WARM uses 127 and CT 454; CINEMA uses 64 and CT 454 when HUE_CINEMA_CT=454 is set. These are approximate brightness fractions, not measured illuminance. Without that setting, the older cinema implementation defaults to an xy colour preset, so preserve the working environment configuration.

6. Experimental procedure

1. Wire and separately test both nodes according to Figure 2. In Arduino IDE select Uno WiFi Rev2 and the correct port. Store Wi-Fi and channel write credentials locally; never include the secrets tab in shared evidence.

2. Upload the final lighting sketch using the phone's 2.4 GHz hotspot credentials. Upload the plant sketch with its home-network credentials. Confirm new ThingSpeak entries and plausible raw measurements.

3. On the Mac, load the local .env in the project directory. Run both read-only readers and inspect age/freshness. Initialise the web login with tools/setup_web.py if necessary.

4. Run one controller instance using the command below. Stop older manual_demo.py or controller processes first to avoid competing commands. Keep the Mac powered, awake, online and its lid open.

5. Verify automatic light-level behaviour before activity tests during 07–23. Trigger sofa/touch, then screen, then release the screen. Allow roughly one publication interval plus polling delay before judging a response.

6. Send TalkBack commands from a browser on another network; capture the resulting lamp change. Display the plant raw value, timestamp and freshness on the local dashboard. Capture evidence without credentials.

```
set -a
source .env
set +a
caffeinate -i .venv/bin/python thingspeak_controller.py --hue --talkback --web
```

7. Results

Evidence	Recorded observation / meaning
E1: initial light read	Sofa 36; screen 25; light 47; touch 0; age 2 s → no activity, screen up, dark
E2: covered screen + illuminated light	Screen 594; light 556; age 2 s → screen down and bright at original thresholds
E3: physical Hue output	OFF command from automatic policy turned actual lamps off
E4: activity and screen	Sofa caused warm light; revised screen threshold >200 produced cinema response
E5: hotspot	User confirmed real lamp response while Node 1 used mobile hotspot
E6: plant dashboard	Raw 598; age 62 s; "Fresh measurement"; also reader raw 597 at age 24 s

Table 4. Session evidence summary. These are transcribed observations; add labelled screenshots/photos and video references to substantiate them.

One confirmed dashboard timestamp was 2026-11-02T22:24:51Z (UTC), equivalent to 23:24:51 in Oslo on that date. A returned moisture value is not a calibrated moisture percentage. Initial manual observations of 0 in dry soil and approximately 380 after watering do not establish a general calibration; subsequent values near 598 belong to later measurements.

8. Testing and evaluation

Test	Expected / observed	Status
Channel data and validation	Recent, range-checked fields; 2–26 s ages shown in samples	Observed
Sofa activity	Warm light at ~50%; user confirmed lamps changed	Observed
Screen >200	Cinema response after lowering threshold; user confirmed	Observed
Hotspot chain	Remote node controls home lamps; user confirmed	Observed
Soil on web page	Raw value, timestamp, age and fresh label displayed	Observed
Screen overrides manual	Expected to cancel override; no isolated final test recorded	To verify
Touch-only activity	Input previously tested; complete final Hue path not isolated	To verify
30 min hold / 15 min expiry	Implemented timers; no timed final run recorded	To verify
Stale feed and reconnection	Hold output >75 s stale; recover on fresh data	To verify
Old TalkBack commands	Intended rejection; final consumer not re-inspected	To verify
Quantitative latency / loss	No timestamped trial series or loss measurement	To measure

Table 5. Expected versus observed behaviour. "Observed" means user-reported physical evidence or supplied output, not an independent laboratory repeat.

Timing model: 20 s publishing and 5 s polling give approximately 0–25 s scheduling delay, with an ideal mean of 12.5 s for uniformly timed changes. HTTPS, Wi-Fi retries, blocking calls and cloud updates add delay; 25 s is not a guaranteed maximum. The sampled data ages demonstrate feed freshness only, not end-to-end actuation latency.

For quantitative evaluation, record at least ten trials with event time, ThingSpeak created_at, controller receive time and visible lamp-change time. Report median, range, failures and trial count. Also record the source and Mac clocks' synchronisation. To measure missing uploads, collect a timed observation window and compare received entries with expected attempts, allowing for reconnects; do not infer packet loss from the samples in this report.

Failures and fixes: incorrect Sharp cable identification caused upload failures; corrected power wiring restored operation. The initial >400 screen threshold responded poorly at the chosen demonstration position, so it was reduced to >200 and a physical response was confirmed. The web server initially failed because instance/web.json was absent; tools/setup_web.py created the login configuration. A standalone --web command was a shell usage error; the flag belongs to the full Python invocation.

9. Discussion

The prototype demonstrates practical remote sensing and actuation across two networks. Its main dependencies are the hotspot, cloud service, Mac process and home network. Holding the last light state on stale telemetry avoids abrupt changes but can leave lights on or off indefinitely. Shutdown, reconnect, malformed-feed and stale-command tests are needed before claiming robust fault tolerance. The controller records Hue errors in Hub status; silence in the console alone does not prove a Hue request succeeded.

Sharp measurements are nonlinear and are used as raw thresholds, not precise distance measurements. The final screen threshold should be checked at the installed positions with repeated screen-up/down samples; hysteresis or multiple-sample filtering could reduce toggling. Grove light values are not lux. The resistive plant sensor can corrode during prolonged insertion and depends on soil conditions; keep only the sensing electrodes in soil and consider intermittent power or a capacitive replacement.

Secrets remain in local .env and Arduino secrets files. The previously visible .env.save backup may also contain credentials and must be excluded from Git and submitted archives. The web server uses loopback binding, password hashing, sessions and CSRF protection; a local address is not remote hosting. Read keys remain server-side. HTTPS is used for ThingSpeak, and Hue certificate/hostname checks were previously confirmed; do not claim a separate TLS validation audit of the final Arduino firmware. Do not include pairing files, key-bearing screenshots or unsanitised firmware in the report package.

The final ThingSpeak path bypasses the old combined light/soil history schema and does not record the new readings in that SQLite history. The plant reader is invoked while rendering the web page, so a failed read displays an error rather than a cached historical result and may delay page rendering. The page refreshes every 30 s; plant data older than 180 s are labelled stale. Moisture never changes lighting policy.

10. Conclusion

The implemented prototype controlled Hue lamps from a remote Arduino using a phone hotspot and displayed independent raw soil-moisture data on an authenticated local dashboard. User-confirmed physical tests demonstrated activity lighting and improved screen detection after changing the threshold to 200. The core objective was demonstrated, while repeatable latency measurements, complete fault/timeout testing, final source capture and remote dashboard access remain outstanding. The next engineering improvement is quantitative validation and robust threshold filtering rather than adding automatic watering.

11. References

[1] Arduino, UNO WiFi Rev2 documentation: https://docs.arduino.cc/hardware/uno-wifi-rev2/

[2] Arduino, WiFiNINA library: https://docs.arduino.cc/libraries/wifinina/

[3] MathWorks, ThingSpeak Arduino library: https://github.com/mathworks/thingspeak-arduino

[4] MathWorks, ThingSpeak documentation (channels, REST API and TalkBack): https://www.mathworks.com/help/thingspeak/

[5] Sharp, GP2Y0A21YK0F datasheet (confirm exact installed part): https://global.sharp/products/device/lineup/data/pdf/datasheet/gp2y0a21yk_e.pdf

[6] Seeed Studio, Grove Moisture Sensor: https://wiki.seeedstudio.com/Grove-Moisture_Sensor/

[7] Philips Hue, developer documentation: https://developers.meethue.com/

[8] Project repository: https://github.com/fargekatten/Water-and-light-system

[9] Supplied IoT_Lab_Report_Template.docx and project-session evidence, 2 November 2026.

Manufacturer links are reference targets, not a claim that every page or installed part was independently checked during preparation. Verify source titles, availability, access dates and the course’s required citation format before submission.

12. Appendices — submission completion pack

A. Evidence and authorship to insert

• Replace the cover placeholders with course, group, names, date and true contributions. Do not invent contributions.

• Insert one clear prototype photo showing board labels, sensor positions and supply/common-ground wiring. Number and caption it as Figure 3.

• Insert readable screenshots for light feed, actual lamp states, local dashboard and hotspot test. Link video timestamps to test rows. Do not reveal keys.

• Add ten or more measured latency trials, a stale-data/recovery test, override-expiry tests and explicit manual-versus-screen priority evidence.

• State use of AI assistance for drafting/illustrations/code if required by the institution; describe work accurately and retain group responsibility for validation.

B. Final source-code package to collect

The complete working source cannot be reconstructed reliably from the report alone. Export final files from the Mac and Arduino IDE PC, not just the older GitHub commit. The local environment is not copied into this document.

Source location	Include without secrets
Arduino IDE PC	Final lighting .ino; final plant .ino; secrets example with placeholders; board/library versions
Mac: new files	thingspeak_lights.py, thingspeak_moisture.py, thingspeak_controller.py
Mac: modified files	lighting.py, hub.py, webapp.py, templates/dashboard.html
Mac: dependencies	control_hue.py, talkback.py, settings.py, history_store.py, tools/setup_web.py, requirements.txt, static assets and login template
Provenance	Final git status / commit; explicit local file list; package and tool versions

Table 6. Required source inventory. New local files were not confirmed committed at the end of the session.

C. Configuration names — values omitted

Mac: LIGHTS_CHANNEL_ID, LIGHTS_READ_API_KEY
MOISTURE_CHANNEL_ID, MOISTURE_READ_API_KEY
HUE_BRIDGE_HOST, HUE_USERNAME, HUE_CINEMA_CT=454
TalkBack settings: confirm names in final talkback.py
Light Arduino: SECRET_SSID, SECRET_PASS, LIGHTS_CHANNEL_ID, SECRET_TS_WRITE_API_KEY
Plant Arduino: network credentials, channel ID, write API key

Exclude .env, .env.save, arduino_secrets.h, instance/hue_pairing.json, instance/web.json, backup files and any copied firmware containing literal credentials. A channel ID is not a password, but API keys are secrets. Share placeholder examples and setup instructions instead.